

Documentación del Proyecto

Práctica de Laboratorio: Monitor Ambiental con Raspberry Pi

1. Introducción

El presente proyecto consiste en el desarrollo de un monitor ambiental utilizando una Raspberry Pi, un sensor DHT11 y una pantalla OLED SSD1306. El sistema permite medir la temperatura y humedad del ambiente en tiempo real, mostrando los valores directamente en la pantalla OLED junto con iconos representativos para facilitar la visualización de la información.

Este proyecto está orientado al aprendizaje de sensores, comunicación I2C, programación modular en Python y visualización de datos embebidos.

2. Objetivo General

Configurar y utilizar un sensor DHT11 para medir temperatura y humedad, visualizar los datos en tiempo real en una pantalla OLED y representar gráficamente la información mediante iconos.

3. Objetivos Específicos

- Configurar la Raspberry Pi para trabajar con sensores y pantallas OLED.
 - Leer datos ambientales utilizando el sensor DHT11.
 - Mostrar la información obtenida en una pantalla OLED SSD1306.
 - Aplicar programación modular en Python.
 - Integrar imágenes e iconos para mejorar la interfaz visual.
 - Implementar buenas prácticas de desarrollo de software embebido.
-

4. Materiales Utilizados

Cantidad Componente

1	Raspberry Pi (3B+, 4 o Zero W)
1	Sensor DHT11
1	Pantalla OLED 128x64 SSD1306
1	Protoboard
Varios	Jumpers
1	Fuente de alimentación 5V
2	Íconos PNG/JPG de temperatura y humedad

5. Requisitos Previos

Antes de iniciar el proyecto se requiere:

- Raspberry Pi OS instalado (Lite o Desktop).
 - Python 3.7 o superior.
 - Acceso a internet.
 - SSH habilitado (opcional).
 - I2C habilitado en Raspberry Pi.
-

6. Instalación de Dependencias

Actualizar el sistema:

```
sudo apt update && sudo apt install -y python3-pip
```

Instalar librerías necesarias:

```
pip3 install adafruit-circuitpython-dht
```

```
pip3 install adafruit-circuitpython-ssd1306
```

```
pip3 install pillow
pip3 install matplotlib

Instalar soporte GPIO:

sudo apt install libgpod2
```

7. Diagrama de Conexión

Sensor DHT11

DHT11 Raspberry Pi

VCC 5V

GND GND

DATA GPIO17

Pantalla OLED SSD1306 (I2C)

OLED Raspberry Pi

VCC 3.3V

GND GND

SCL GPIO3 (SCL)

SDA GPIO2 (SDA)

8. Estructura del Proyecto

```
dht11-oled/
|
├── main.py
├── sensors/
|   └── dht11_reader.py
```

```
└─ display/
|   └─ oled_display.py
|   └─ icons/
|       └─ temp.png
|       └─ humidity.png
└─ utils/
    └─ data_logger.py
```

9. Descripción del Funcionamiento

El sistema trabaja de manera continua siguiendo los siguientes pasos:

1. El sensor DHT11 captura la temperatura y humedad del ambiente.
 2. Los datos son procesados por la Raspberry Pi mediante Python.
 3. La pantalla OLED muestra:
 - Temperatura en grados Celsius.
 - Humedad relativa en porcentaje.
 - Íconos representativos.
 4. El proceso se repite cada 2 segundos.
-

10. Explicación del Código

10.1 Archivo: dht11_reader.py

Este módulo se encarga de leer la información proveniente del sensor DHT11.

Funciones principales:

- Inicializar el sensor.
- Obtener temperatura y humedad.
- Manejar errores de lectura.

Código:

```
import adafruit_dht
import board
import time

class DHT11Reader:
    def __init__(self, pin=board.D17):
        self.sensor = adafruit_dht.DHT11(pin)

    def read(self):
        try:
            temp = self.sensor.temperature
            hum = self.sensor.humidity
            return temp, hum
        except RuntimeError:
            return None, None
```

10.2 Archivo: oled_display.py

Controla la pantalla OLED y muestra los datos con iconos.

Funciones principales:

- Inicializar pantalla OLED.
- Cargar imágenes.
- Mostrar temperatura y humedad.

Código:

```
from PIL import Image, ImageDraw, ImageFont
import board, busio, adafruit_ssd1306
import time, os

class OLEDDisplay:
    def __init__(self, width=128, height=64):
```

```
i2c = busio.I2C(board.SCL, board.SDA)
self.disp = adafruit_ssd1306.SSD1306_I2C(width, height, i2c)
self.width = width
self.height = height
self.disp.fill(0)
self.disp.show()
self.font = ImageFont.load_default()
self.load_icons()
```

```
def load_icons(self):
```

```
    base = os.path.dirname(__file__)
    self.temp_icon = Image.open(
        os.path.join(base, "icons", "temp.png")
    ).resize((16,16)).convert("1")
```

```
    self.hum_icon = Image.open(
        os.path.join(base, "icons", "humidity.png")
    ).resize((16,16)).convert("1")
```

```
def update(self, temperature, humidity):
```

```
    image = Image.new("1", (self.width, self.height))
    draw = ImageDraw.Draw(image)
```

```
    image.paste(self.temp_icon, (0, 0))
    draw.text((20, 0), f"{temperature}°C",
        font=self.font, fill=255)
```

```
    image.paste(self.hum_icon, (0, 20))
    draw.text((20, 20), f"{humidity}%",
        font=self.font, fill=255)
```

```
    self.disp.image(image)
    self.disp.show()
```

10.3 Archivo: main.py

Archivo principal que ejecuta todo el sistema.

Código:

```
import time
from sensors.dht11_reader import DHT11Reader
from display.oled_display import OLEDDisplay

def main():
    sensor = DHT11Reader()
    display = OLEDDisplay()

    while True:
        temp, hum = sensor.read()

        if temp is not None and hum is not None:
            display.update(temp, hum)

        time.sleep(2)

if __name__ == "__main__":
    main()
```

11. Implementación de Imágenes

Para los iconos se recomienda utilizar imágenes:

- Formato PNG, JPG o ICO.
- Tamaño de 16x16 píxeles.
- Fondo simple o transparente.

Conversión de imágenes

Se recomienda utilizar el sitio:

Copiar imágenes vía SSH

Desde la computadora:

```
scp *.png USUARIO@ipRPi:~
```

Dentro de la Raspberry Pi: `mv *.png dht11-oled/display/icons/`

12. Ejecución del Proyecto

Entrar a la carpeta del proyecto:

```
cd dht11-oled
```

Ejecutar el programa:

```
python3 main.py
```

13. Resultados Esperados

Al ejecutar el sistema correctamente se deberá observar:

- Temperatura en tiempo real.
 - Humedad relativa.
 - Actualización automática cada 2 segundos.
 - Iconos representativos en la pantalla OLED.
-

14. Posibles Extensiones

El proyecto puede ampliarse con nuevas funcionalidades como:

- Almacenamiento de datos en archivos CSV.
- Envío de información a plataformas IoT.

- Integración con MQTT o ThingSpeak.
 - Alertas automáticas.
 - Interfaz web con Flask o Dash.
 - Gráficas históricas con Matplotlib.
-

15. Buenas Prácticas Implementadas

Durante el desarrollo se aplicaron las siguientes buenas prácticas:

- Código modular.
 - Separación de responsabilidades.
 - Manejo de excepciones.
 - Organización profesional de carpetas.
 - Reutilización de código.
 - Escalabilidad del sistema.
-

16. Conclusión

La práctica permitió implementar un sistema de monitoreo ambiental funcional utilizando Raspberry Pi y sensores electrónicos. Se integraron conceptos de electrónica, programación embebida y visualización de datos en tiempo real.

Además, se aplicaron técnicas modernas de desarrollo modular, facilitando la ampliación y mantenimiento futuro del proyecto. El sistema desarrollado puede utilizarse como base para aplicaciones de automatización, domótica e Internet de las Cosas (IoT).

